LAPORAN TUGAS INDIVIDU

Mata Kuliah Sistem Operasi (USA-WP2260110)

Pemetaan Sistem Operasi pada Ekosistem IoT dan Smart Farming

Keterangan	Isi
Kode dan judul tugas	I-1 Pemetaan Sistem Operasi pada Ekosistem IoT dan Smart Farming
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-1 / 29 Oktober 2026
Kepingan puzzle	Kepingan #1 → Level 1 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan **3**
1.1 Latar belakang kasus 3
1.2 Tujuan tugas 3
1.3 Ruang lingkup 3
2. Dasar Teori Singkat **3**
3. Lingkungan Praktik **4**
4. Langkah Pengerjaan **5**
4.1 Langkah 1: Mencatat identitas sistem operasi 5
4.2 Langkah 2: Membaca perangkat keras yang tersedia 5
4.3 Langkah 3: Mengukur pemakaian memori 6
4.4 Langkah 4: Memeriksa layanan dan proses 6
4.5 Langkah 5: Menjalankan program pemetaan ekosistem 7
4.6 Langkah 6: Menggambar peta ekosistem 8
5. Hasil dan Analisis **8**
5.1 Hasil 8
5.2 Analisis 10
5.3 Catatan keterbatasan 11
6. Kesimpulan **11**
7. Kepingan Puzzle **12**
Daftar Pustaka **12**
Lampiran A. Kode Program dan Log **13**
Lampiran B. Pernyataan Penggunaan AI Generatif **17**
Lampiran C. Daftar Periksa Sebelum Mengumpulkan **17**
Swa-nilai berdasarkan kriteria lembar tugas 18

1. Pendahuluan

1.1 Latar belakang kasus

Sistem pertanian cerdas bukan satu komputer, melainkan rangkaian perangkat yang saling terhubung: mikrokontroler di petak lahan yang membaca kelembapan tanah, gateway di kebun yang mengumpulkan data, server di cloud yang menyimpan riwayat, dan ponsel petani yang menampilkan kondisi lahan sekaligus menyalakan pompa. Setiap perangkat itu menjalankan sistem operasi yang berbeda jenis, dengan kemampuan yang jauh berbeda pula.

Perbedaan itu bukan hal sepele. Mikrokontroler ESP32 hanya memiliki sekitar 520 KB SRAM dan tidak punya unit manajemen memori, sehingga sistem operasinya harus muat dalam ruang sekecil itu. Sebaliknya, server cloud menjalankan sistem operasi lengkap dengan memori puluhan gigabita dan puluhan proses berjalan bersamaan. Memilih sistem operasi yang tidak sesuai dengan kemampuan perangkat akan membuat sistem gagal bekerja sebelum sempat diuji. Karena itu, memetakan sistem operasi pada tiap lapisan ekosistem adalah langkah pertama sebelum merancang sistem apa pun di atasnya.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-1 (CPMK042, CPMK043, CPMK081, CPMK101, CPMK111). Dijabarkan menjadi tiga tujuan:

- **Pertama,** mengklasifikasikan jenis sistem operasi pada setiap perangkat ekosistem IoT pertanian ke dalam empat kategori: desktop, server, mobile, dan embedded/RTOS;
- **Kedua,** menjelaskan fungsi utama sistem operasi pada tiap perangkat, dikaitkan dengan keterbatasan perangkat kerasnya;
- **Ketiga,** menyajikan peta ekosistem sistem operasi dalam satu diagram yang memperlihatkan hubungan antar-lapisan beserta arah aliran datanya.

1.3 Ruang lingkup

Pengukuran dilakukan langsung pada komputer operator yang dipakai menyusun laporan, yaitu mesin Linux dengan nama host INFINIX-ElementaryOS. Perangkat lain pada ekosistem — mikrokontroler ESP32, gateway Raspberry Pi, server cloud, dan ponsel Android — dipetakan berdasarkan dokumen resmi pabriknya, bukan diukur langsung, karena tidak tersedia di tempat praktik.

Yang tidak dikerjakan: pengukuran konsumsi daya, pengujian pada perangkat keras sesungguhnya, dan pembahasan keamanan sistem operasi.

2. Dasar Teori Singkat

Sistem operasi adalah perangkat lunak sistem yang mengendalikan seluruh sumber daya komputer dan menyediakan landasan bagi program aplikasi (Helmiawan, t.t.). Tanpa lapisan ini, pemrogram harus berhadapan langsung dengan kerumitan perangkat keras: mengatur letak data di memori, menentukan giliran prosesor, dan menangani perangkat masukan-

keluaran satu per satu. Sistem operasi menyembunyikan kerumitan itu di balik antarmuka yang seragam (Silberschatz dkk., 2018).

Fungsi utama sistem operasi dapat diringkas menjadi lima: mengatur penjadwalan proses, mengelola memori, mengelola sistem berkas, mengendalikan perangkat masukan-keluaran, dan menyediakan mekanisme perlindungan antar-proses. Tidak semua fungsi itu hadir dengan bobot yang sama pada setiap perangkat. Pada server, kelima fungsi berjalan penuh dengan banyak pengguna sekaligus. Pada mikrokontroler, yang ada hanyalah penjadwal sederhana dan pengelola memori statis, karena memang tidak ada disk maupun banyak pengguna.

Klasifikasi menurut perangkat. Sistem operasi dibedakan berdasarkan sasaran perangkat keras dan cara pemakaiannya:

- **a.** Desktop, dipakai satu pengguna dengan antarmuka grafis penuh, contohnya elementary OS dan Windows.
- **b.** Server, dijalankan tanpa layar (headless), melayani banyak permintaan jaringan, contohnya Ubuntu Server LTS.
- **c.** Mobile, berjalan pada ponsel dengan pembatasan ketat per aplikasi, contohnya Android dan iOS.
- **d.** Embedded/RTOS, ditanam pada perangkat dengan sumber daya kecil dan menuntut waktu respons yang dapat diprediksi, contohnya FreeRTOS pada ESP32.

Kategori keempat itulah yang paling berbeda sifatnya. Sistem operasi waktu nyata (real-time operating system) tidak mengejar rata-rata waktu respons tercepat, melainkan jaminan bahwa tugas tertentu selesai dalam batas waktu yang sudah ditentukan. Pada ESP32, FreeRTOS menjalankan penjadwal berbasis prioritas dengan pengalihan tugas dalam hitungan mikrodetik (FreeRTOS Project, 2024). Jaminan semacam itu tidak bisa diberikan sistem operasi desktop, yang justru mengorbankan kepastian demi pemanfaatan prosesor yang merata.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS, RAM 15216 MiB, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — pengukuran dijalankan langsung pada mesin host
VM / host	INFINIX-ElementaryOS, 16 CPU logis, penyimpanan 464 GiB
Perangkat lain	Tidak tersedia — ESP32, Raspberry Pi, server cloud, dan ponsel dipetakan dari dokumen resmi pabrik
Bahasa dan alat	Python 3.12.3; perintah hostname, uname, lscpu, df, systemctl, ps; GNOME Shell Screenshot untuk bukti tangkapan layar
Berkas semu	/proc/meminfo untuk memori, /proc/1 untuk proses init, /sys/fs/cgroup untuk hierarki kontrol sumber daya

Satu hal yang perlu dicatat: laporan ini tidak memakai mesin virtual. Lembar tugas I-1 tidak mensyaratkannya, dan seluruh pengukuran yang dibutuhkan — identitas sistem, jumlah CPU, kapasitas memori, serta layanan yang berjalan — dapat dibaca langsung dari mesin host. Mesin virtual baru diperlukan pada tugas yang menuntut eksperimen berisiko atau

pengamatan yang harus dapat diulang persis.

4. Langkah Pengerjaan

Pengumpulan bukti dibagi menjadi enam langkah. Seluruh perintah dijalankan pada terminal mesin host, dan setiap tangkapan layar memperlihatkan prompt beserta perintah yang menghasilkan keluarannya. Skrip ambil-bukti.sh menjalankan urutan ini secara otomatis agar hasilnya dapat diulang.

4.1 Langkah 1: Mencatat identitas sistem operasi

Langkah pertama memastikan sistem apa yang sebenarnya sedang diukur. Nama host, versi kernel, dan distribusi dicatat karena ketiganya menentukan fasilitas apa yang tersedia — misalnya ketersediaan cgroup versi 2 atau cara pembacaan /proc.

```
$ hostname
INFINIX-ElementaryOS

$ uname -srm
Linux 7.0.0-34-generic x86_64

$ grep -E "PRETTY_NAME|VERSION_ID" /etc/os-release
PRETTY_NAME="elementary OS 8"
VERSION_ID="8"
UBUNTU_PRETTY_NAME="Ubuntu 24.04.5 LTS"
```



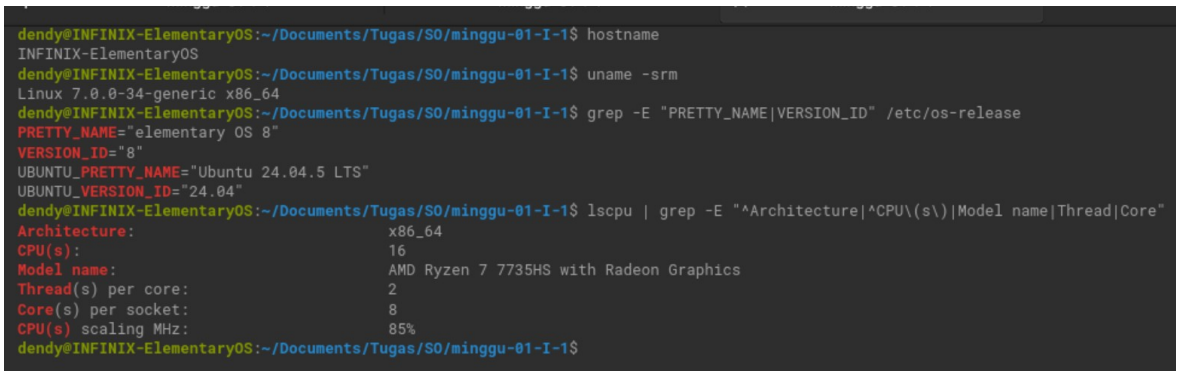
Gambar 4.1 Prompt, perintah, dan keluaran identitas sistem pada terminal host

4.2 Langkah 2: Membaca perangkat keras yang tersedia

Jumlah prosesor dan kapasitas memori menentukan seberapa besar beban yang bisa ditangani mesin ini sebagai simpul operator. Angka CPU dibaca dari lscpu, memori dari /proc/meminfo, dan penyimpanan dari df.

```
$ lscpu | grep -E "^Architecture|^CPU\(s\)|Model name|Thread|Core"
Architecture:            x86_64
CPU(s):                  16
Model name:              AMD Ryzen 7 7735HS with Radeon Graphics
Thread(s) per core:      2
Core(s) per socket:      8

$ df -hT /
Filesystem             Type  Size  Used Avail Use% Mounted on
/dev/mapper/data-root ext4  464G  150G  314G  32% /
```



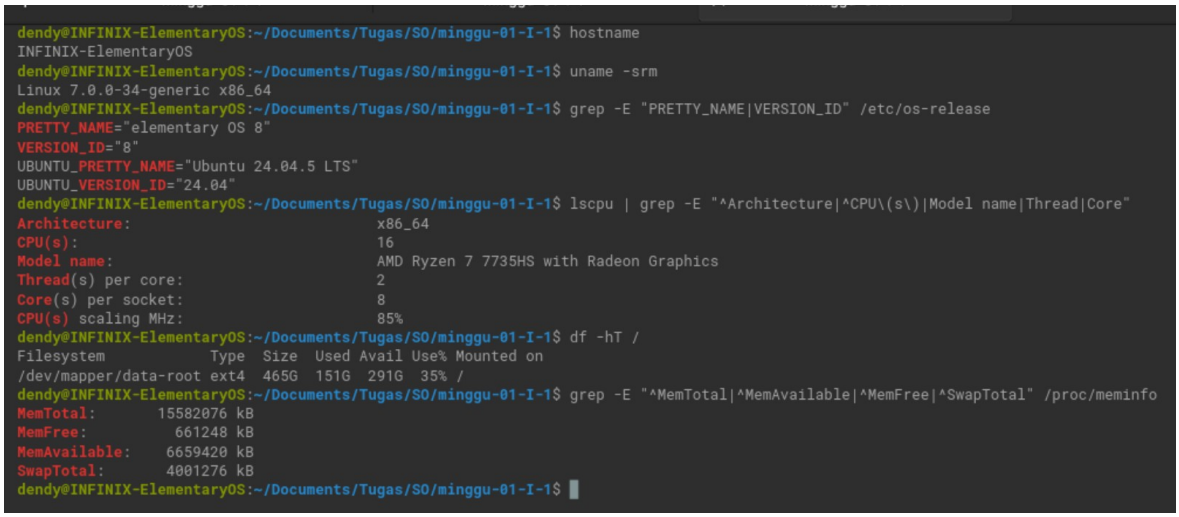
Gambar 4.2 Keluaran lscpu dan df yang memperlihatkan CPU serta sistem berkas ext4

4.3 Langkah 3: Mengukur pemakaian memori

MemTotal menunjukkan kapasitas terpasang, sedangkan MemAvailable menunjukkan perkiraan memori yang benar-benar bisa dipakai proses baru — angka ini lebih jujur daripada MemFree, karena cache dan buffer sebenarnya dapat diambil kembali oleh kernel bila dibutuhkan.

```
$ grep -E "^MemTotal|^MemAvailable|^MemFree|^SwapTotal" /proc/meminfo
MemTotal:       15582076 kB
MemFree:          688332 kB
MemAvailable:    6686784 kB
SwapTotal:       4001276 kB
```

Keluaran pembacaan /proc/meminfo



Gambar 4.3 Pembacaan /proc/meminfo beserta perintah sebelumnya dalam satu sesi

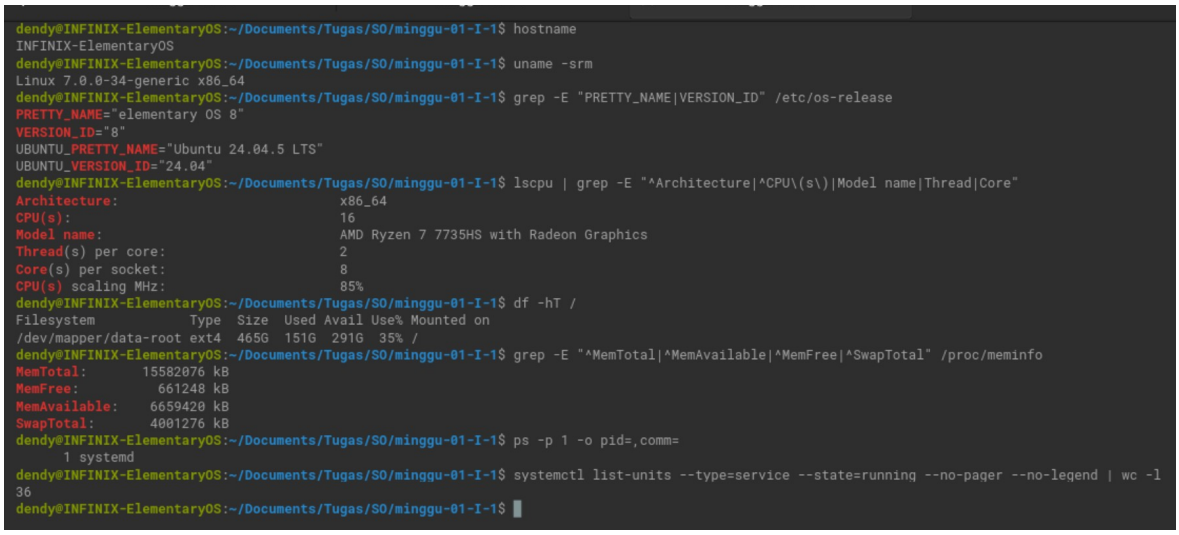
4.4 Langkah 4: Memeriksa layanan dan proses

Sistem operasi desktop tetap menjalankan banyak layanan latar belakang. Jumlah unit systemd yang berjalan dan jumlah proses keseluruhan memberi gambaran seberapa besar beban dasar sebelum aplikasi pengguna dijalankan. Proses init diperiksa untuk memastikan mekanisme mana yang mengelola layanan.

```
$ ps -p 1 -o pid=,comm=
      1 systemd

$ systemctl list-units --type=service --state=running --no-pager --no-legend | wc -l
36

$ ps -e --no-headers | wc -l
501
```



Gambar 4.4 Proses init systemd dan jumlah unit layanan yang berjalan

4.5 Langkah 5: Menjalankan program pemetaan ekosistem

Program peta_so_kebun.py membaca langsung /proc/meminfo dan /etc/os-release, sehingga angka yang dicetak selalu sesuai mesin yang menjalankannya. Program juga menghitung perkiraan volume data dari dua belas node sensor dengan jeda lima detik, sebagai bahan pertimbangan beban penyimpanan pada gateway.

```
$ python3 peta_so_kebun.py
=== PEMETAAN EKOSISTEM SO SMART FARMING ===
Mesin pengukuran : INFINIX-ElementaryOS
Distro           : elementary OS 8
Kernel           : 7.0.0-34-generic (x86_64)
Python           : 3.12.3
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ hostname
INFINIX-ElementaryOS
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ uname -srm
Linux 7.0.0-34-generic x86_64
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ grep -E "PRETTY_NAME|VERSION_ID" /etc/os-release
PRETTY_NAME="elementary OS 8"
VERSION_ID="8"
UBUNTU_PRETTY_NAME="Ubuntu 24.04.5 LTS"
UBUNTU_VERSION_ID="24.04"
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ lscpu | grep -E "^Architecture|^CPU\(s\)|Model name|Thread|Core"
Architecture:                         x86_64
CPU(s):                               16
Model name:                           AMD Ryzen 7 7735HS with Radeon Graphics
Thread(s) per core:                   2
Core(s) per socket:                   8
CPU(s) scaling MHz:                   85%
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ df -hT /
Filesystem           Type  Size  Used Avail Use% Mounted on
/dev/mapper/data-root ext4  465G  151G  291G  35% /
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ grep -E "^MemTotal|^MemAvailable|^MemFree|^SwapTotal" /proc/meminfo
MemTotal:       15582076 kB
MemFree:          661248 kB
MemAvailable:    6659420 kB
SwapTotal:       4001276 kB
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ ps -p 1 -o pid=,comm=
      1 systemd
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ systemctl list-units --type=service --state=running --no-pager --no-legend | wc -l
36
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ ps -e --no-headers | wc -l
501
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$ python3 peta_so_kebun.py
=== PEMETAAN EKOSISTEM SO SMART FARMING ===
Mesin pengukuran : INFINIX-ElementaryOS
Distro           : elementary OS 8
Kernel           : 7.0.0-34-generic (x86_64)
Python           : 3.12.3

Perangkat                       Sistem operasi          Jenis          Fungsi utama
-------------------------------------------------------------------------------------------------------------
Mikrokontroler sensor (ESP32)   FreeRTOS                Embedded/RTOS  Penjadwalan deterministik, RAM ratusan KB, tanpa MMU
Gateway kebun (Raspberry Pi)    Raspberry Pi OS Lite    Server         Headless, meneruskan data sensor, buffer lokal
Server cloud pertanian          Ubuntu Server LTS       Server         Agregasi multi-gateway, basis data, TLS
Ponsel pemantau petani          Android                 Mobile         Sandbox UID, notifikasi, kontrol pompa
Komputer operator (mesin ini)   elementary OS (Linux)   Desktop        Alat ukur dan dasbor operator

=== PENGUKURAN SUMBER DAYA (data nyata) ===
MemTotal         : 15582076 kB (15216.9 MiB)
MemAvailable     : 6686784 kB (6530.1 MiB)
Terpakai         : 8686.8 MiB (57.09% dari total)
SwapTotal        : 4001276 kB
CPU logis        : 16
Penyimpanan /    : 150 GiB terpakai dari 464 GiB

=== PERKIRAAN BEBAN 12 NODE SENSOR ===
Jeda pembacaan   : 5 detik
Baris per node   : 17280 baris/hari
Total baris/hari : 207360 baris/hari
Volume data/hari : 31.64 MiB/hari
Volume 30 hari   : 949.2 MiB

tersimpan: results/peta.json
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-01-I-1$
```

Gambar 4.5 Keluaran lengkap peta_so_kebun.py beserta perhitungan beban 12 node

4.6 Langkah 6: Menggambar peta ekosistem

Data dari langkah sebelumnya diubah menjadi dua gambar oleh figures.py: peta ekosistem lima simpul, dan grafik pemakaian memori. Peta disusun dua kolom untuk memisahkan perangkat di sisi lahan dari layanan terpusat, dengan panah dua arah untuk membedakan aliran data sensor dari aliran perintah aktuator.

```
$ ../.venv/bin/python figures.py
gambar: ['gambar-memori.png', 'gambar-peta-ekosistem.png']
```

5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 menyajikan klasifikasi jenis dan fungsi sistem operasi pada lima simpul ekosistem. Kolom jenis memakai empat kategori yang diminta lembar tugas.

No	Perangkat	Sistem operasi	Jenis	Fungsi utama
1	Mikrokontroler sensor ESP32 (12 node)	FreeRTOS	Embedded/ RTOS	Penjadwalan deterministik, RAM sekitar 520 KB, tanpa MMU

No	Perangkat	Sistem operasi	Jenis	Fungsi utama
2	Gateway kebun Raspberry Pi	Raspberry Pi OS Lite	Server	Headless, meneruskan data sensor, menyimpan buffer saat jaringan putus
3	Server cloud pertanian	Ubuntu Server LTS	Server	Agregasi banyak gateway, basis data, TLS
4	Ponsel pemantau petani	Android	Mobile	Sandbox per aplikasi, notifikasi, kendali pompa
5	Komputer operator (mesin ukur)	elementary OS 8	Desktop	Alat ukur, penyusunan laporan, dasbor operator

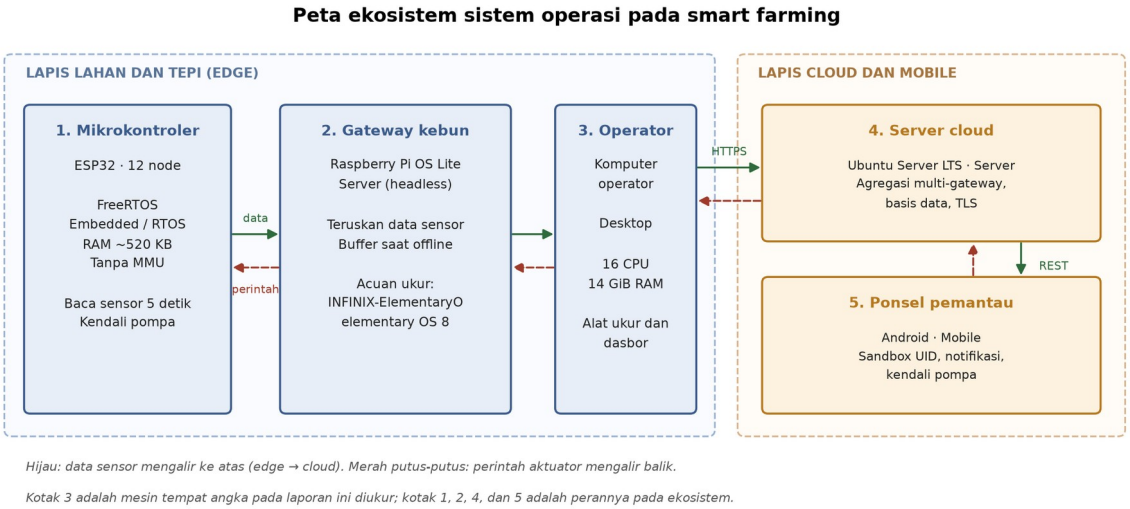
Tabel 5.1 Klasifikasi jenis dan fungsi sistem operasi pada ekosistem smart farming

Tabel 5.2 memuat angka yang diukur langsung pada mesin operator. Kolom sumber memisahkan mana yang hasil ukur dan mana yang berasal dari dokumen pabrik, supaya tidak tercampur.

No	Yang diukur	Hasil	Sumber
1	Memori terpasang (MemTotal)	15582076 kB (15217 MiB)	Hasil ukur /proc/meminfo
2	Memori tersedia (MemAvailable)	6686784 kB (6530 MiB)	Hasil ukur /proc/meminfo
3	Memori terpakai	8687 MiB (57.1% dari total)	Hitungan dari baris 1 dan 2
4	Swap terpasang	4001276 kB	Hasil ukur /proc/meminfo
5	CPU logis	16 thread (8 core, 2 thread/core)	Hasil ukur lscpu
6	Penyimpanan akar (ext4)	150 GiB terpakai dari 464 GiB	Hasil ukur df -hT
7	Unit layanan systemd berjalan	36 unit	Hasil ukur systemctl
8	Jumlah proses	501 proses	Hasil ukur ps -e
9	Volume data 12 node sensor	207360 baris/hari, 31.64 MiB/hari	Hitungan program (jeda 5 detik, 160 byte/baris)
10	Kapasitas RAM ESP32	sekitar 520 KB SRAM	Dokumen pabrik Espressif

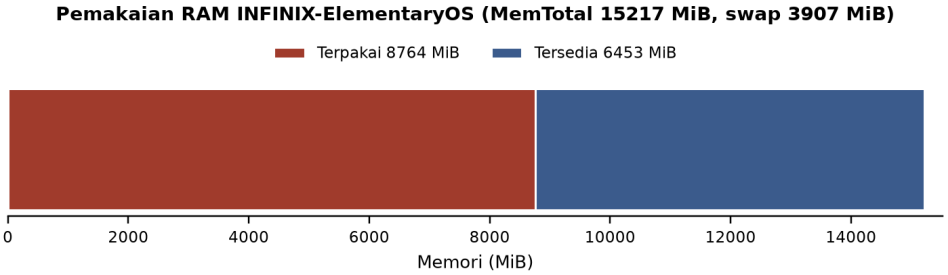
Tabel 5.2 Hasil ukur pada mesin operator dan acuan dari dokumen pabrik

Gambar 5.1 memperlihatkan peta ekosistem kelima simpul beserta arah alirannya.



Gambar 5.1 Peta ekosistem sistem operasi pada IoT dan smart farming

Gambar 5.2 membandingkan memori terpakai dengan yang masih tersedia pada mesin ukur.



Gambar 5.2 Pemakaian RAM pada mesin operator saat pengukuran

5.2 Analisis

Poin (1) deskripsi tugas — klasifikasi jenis. Kelima simpul jatuh ke empat kategori berbeda, dan yang paling menentukan bukan merek sistem operasinya melainkan ketersediaan unit manajemen memori serta jumlah pengguna yang harus dilayani. ESP32 tidak memiliki MMU sama sekali; semua alamat memori bersifat fisik dan sudah ditentukan saat kompilasi. Karena itu FreeRTOS di dalamnya tidak menjalankan proses terpisah seperti Linux, melainkan task yang berbagi satu ruang alamat. Tidak adanya isolasi ini bukan kekurangan, melainkan konsekuensi wajar: dengan SRAM sekitar 520 KB, membuat tabel halaman untuk isolasi memori justru memakan ruang yang tidak tersedia.

Sebaliknya, mesin operator pada laporan ini memiliki 16 thread CPU, memori terpasang 15217 MiB, dan menjalankan 501 proses bersamaan. Pada mesin seperti ini, isolasi antar-proses justru menjadi syarat utama: satu peramban yang kehabisan memori tidak boleh menjatuhkan seluruh sistem. Sistem operasi desktop dan server memakai mekanisme yang sama — MMU, tabel halaman, dan pemisahan ruang alamat — untuk mencapai hal yang tidak mungkin dilakukan pada mikrokontroler.

Menariknya, gateway Raspberry Pi dan server cloud sama-sama masuk kategori server, padahal kemampuan keduanya berbeda jauh. Yang menyatukan keduanya adalah cara pakai: keduanya dijalankan tanpa layar dan melayani permintaan dari perangkat lain. Kategori pada lembar tugas dengan demikian lebih menandai peran dalam arsitektur

daripada ukuran perangkat keras.

Poin (2) deskripsi tugas — fungsi utama pada tiap perangkat. Pada mikrokontroler, fungsi sistem operasi menyempit menjadi penjadwalan dan pengelolaan antrean. FreeRTOS menyediakan penjadwal berbasis prioritas yang dapat diprediksi: tugas pembacaan sensor dijalankan setiap lima detik, dan tugas kendali pompa mendapat prioritas lebih tinggi agar perintah menutup katup tidak tertunda oleh pembacaan sensor. Tidak ada sistem berkas, tidak ada pengguna, dan tidak ada manajemen memori dinamis yang rumit.

Pada gateway, fungsi itu melebar. Raspberry Pi OS Lite menjalankan sistem berkas penuh untuk menyimpan buffer saat jaringan putus, menjalankan layanan jaringan, dan mengelola beberapa proses sekaligus seperti klien MQTT dan agen pengirim. Fungsi manajemen memori mulai penting karena buffer harus dibatasi agar gateway tidak kehabisan memori saat menyimpan data berhari-hari. Perhitungan pada Tabel 5.2 menunjukkan volume 31.6 MiB per hari untuk dua belas node; dalam tiga puluh hari menjadi 949 MiB, angka yang masih wajar untuk kartu microSD kelas biasa, tetapi cukup untuk menuntut kebijakan rotasi berkas.

Pada server cloud dan mesin operator, kelima fungsi sistem operasi berjalan penuh. Di sinilah pengelolaan sumber daya menjadi paling rumit: mesin operator dengan 501 proses hanya memakai 8687 MiB dari 15217 MiB, artinya sekitar 57 persen. Sisa memori bukan terbuang; sebagian besar menjadi cache sistem berkas yang bisa diambil kembali kapan saja. Inilah sebabnya MemAvailable lebih tepat dipakai daripada MemFree saat menilai beban sebenarnya, dan inilah wujud nyata fungsi manajemen memori yang tidak dimiliki mikrokontroler.

Poin (3) deskripsi tugas — peta ekosistem. Pada Gambar 5.1, arah aliran dibedakan menjadi dua: panah hijau membawa data sensor dari lahan menuju cloud, panah merah putus-putus membawa perintah aktuator kembali ke lahan. Pemisahan arah ini penting karena keduanya menuntut jaminan yang berbeda. Data sensor boleh terlambat beberapa detik tanpa akibat serius, sedangkan perintah menutup pompa harus sampai dan dijalankan. Dari sudut pandang sistem operasi, perbedaan itu diterjemahkan menjadi prioritas task yang berbeda di ESP32 dan gateway, bukan sekadar prioritas paket di jaringan.

Lapisan pada peta juga memperlihatkan bahwa sistem operasi yang paling banyak jumlahnya justru yang paling sederhana. Dua belas node sensor menjalankan FreeRTOS, satu gateway menjalankan Raspberry Pi OS, satu server menjalankan Ubuntu Server, dan satu ponsel menjalankan Android. Artinya keseragaman sistem operasi bukan tujuan di arsitektur seperti ini; yang dibutuhkan justru penyesuaian dengan batas kemampuan tiap perangkat, dengan protokol yang menyatukan mereka di lapisan aplikasi.

5.3 Catatan keterbatasan

Empat dari lima simpul pada laporan ini tidak diukur langsung. Angka kapasitas RAM ESP32, kebutuhan memori Raspberry Pi OS, dan persyaratan server diambil dari dokumen resmi pabrik, dan hal itu sudah ditandai pada kolom sumber Tabel 5.2. Pembacaan langsung hanya berlaku untuk mesin operator. Pembaca yang ingin memverifikasi angka perangkat lain dapat mengulang pengukuran yang sama pada perangkat sesungguhnya dengan perintah yang dipakai di Bagian 4.

6. Kesimpulan

Ekosistem IoT pertanian memakai empat jenis sistem operasi sekaligus, dan kelima simpul yang dipetakan jatuh ke kategori embedded/RTOS, server, mobile, dan desktop. Yang membedakan mereka bukan merek, melainkan ketersediaan MMU dan jumlah pengguna yang harus dilayani: ESP32 tanpa MMU menjalankan FreeRTOS dengan task berbagi ruang alamat, sementara mesin operator dengan 16 thread CPU dan 15217 MiB memori menjalankan 501 proses dengan isolasi penuh. Fungsi sistem operasi menyempit di ujung lahan dan melebar di pusat data. Karena itu peta ekosistem sebaiknya dibaca sebagai pembagian peran, bukan sebagai tingkatan kualitas: pilihan sistem operasi yang tepat adalah yang sesuai dengan batas kemampuan perangkatnya.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#1
Isi kepingan	Tabel klasifikasi jenis dan fungsi sistem operasi pada lima simpul ekosistem, peta ekosistem beserta arah aliran data, dan hasil ukur sumber daya mesin operator
Lokasi di portofolio	https://github.com/ZzDendy/so-i1-peta-ekosistem
Dipakai untuk	Level 1 Puzzle UTS — menjadi dasar kepingan #2 (system call) dan #3 (proses dan thread), karena pembagian peran antar-simpul di sini menentukan jenis pemanggilan sistem dan model proses yang diamati pada tugas berikutnya

Daftar Pustaka

FreeRTOS Project. (2024). The FreeRTOS kernel: Documentation and FAQ. https://www.freertos.org/Documentation

Helmiawan, M. A. (t.t.). Pendahuluan: Modul pembelajaran sistem operasi. Universitas Sebelas April.

Kerrisk, M. (2010). The Linux programming interface. No Starch Press.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Lampiran A. Kode Program dan Log

Dua berkas di bawah ini menghasilkan seluruh angka dan gambar pada laporan. Keduanya dapat dijalankan ulang pada mesin yang sama.

A.peta_so_kebun.py

```
"""Peta ekosistem SO untuk smart farming, dibaca dari mesin tempat skrip berjalan.

Klasifikasi dan diagram tidak ditebak: nilai RAM, kernel, dan jumlah CPU diambil
dari /proc dan /etc, bukan ditulis tangan.
"""
import json
import os
import platform
import shutil

KOMPONEN = [
    ("Mikrokontroler sensor (ESP32)", "FreeRTOS", "Embedded/RTOS",
     "Penjadwalan deterministik, RAM ratusan KB, tanpa MMU"),
    ("Gateway kebun (Raspberry Pi)", "Raspberry Pi OS Lite", "Server",
     "Headless, meneruskan data sensor, buffer lokal"),
    ("Server cloud pertanian", "Ubuntu Server LTS", "Server",
     "Agregasi multi-gateway, basis data, TLS"),
    ("Ponsel pemantau petani", "Android", "Mobile",
     "Sandbox UID, notifikasi, kontrol pompa"),
    ("Komputer operator (mesin ini)", "elementary OS (Linux)", "Desktop",
     "Alat ukur dan dasbor operator"),
]


def baca_meminfo():
    info = {}
    with open("/proc/meminfo", encoding="utf-8") as f:
        for baris in f:
            kunci, nilai = baris.split(":", 1)
            info[kunci.strip()] = int(nilai.strip().split()[0])
    return info


def identitas():
    osr = {}
    with open("/etc/os-release", encoding="utf-8") as f:
        for baris in f:
            if "=" in baris:
                k, v = baris.rstrip().split("=", 1)
                osr[k] = v.strip('"')
    return {
        "hostname": platform.node(),
        "kernel": platform.release(),
        "arsitektur": platform.machine(),
        "distro": osr.get("PRETTY_NAME", "tidak diketahui"),
        "id_distro": osr.get("ID", "?"),
        "python": platform.python_version(),
    }


def hitung_beban(jumlah_node=12, jeda_detik=5, ukuran_baris=160):
    """Perkiraan volume data harian dari sejumlah node sensor."""
    baris_per_node = 86400 // jeda_detik
    total_baris = jumlah_node * baris_per_node
    total_byte = total_baris * ukuran_baris
    return {
        "jumlah_node": jumlah_node,
        "jeda_detik": jeda_detik,
        "baris_per_node": baris_per_node,
        "total_baris": total_baris,
        "data_kib": total_byte / 1024,
        "data_mib": total_byte / (1024 * 1024),
    }


def main():
    ident = identitas()
    mem = baca_meminfo()
    beban = hitung_beban()

    total_mib = mem["MemTotal"] / 1024
    tersedia_mib = mem["MemAvailable"] / 1024
    terpakai_mib = total_mib - tersedia_mib
    persen = terpakai_mib / total_mib * 100
```

```
    disk = shutil.disk_usage("/")
    disk_total_gib = disk.total / 1024**3
    disk_pakai_gib = disk.used / 1024**3

    print("=== PEMETAAN EKOSISTEM SO SMART FARMING ===")
    print(f"Mesin pengukuran : {ident['hostname']}")
    print(f"Distro           : {ident['distro']}")
    print(f"Kernel           : {ident['kernel']} ({ident['arsitektur']})")
    print(f"Python           : {ident['python']}")
    print()
    print(f"{'Perangkat':<34} {'Sistem operasi':<24} {'Jenis':<15} Fungsi utama")
    print("-" * 118)
    for perangkat, so, jenis, fungsi in KOMPONEN:
        print(f"{perangkat:<34} {so:<24} {jenis:<15} {fungsi}")

    print()
    print("=== PENGUKURAN SUMBER DAYA (data nyata) ===")
    print(f"MemTotal         : {mem['MemTotal']} kB ({total_mib:.1f} MiB)")
    print(f"MemAvailable     : {mem['MemAvailable']} kB ({tersedia_mib:.1f} MiB)")
    print(f"Terpakai         : {terpakai_mib:.1f} MiB ({persen:.2f}% dari total)")
    print(f"SwapTotal        : {mem.get('SwapTotal', 0)} kB")
    print(f"CPU logis        : {os.cpu_count()}")
    print(f"Penyimpanan /    : {disk_pakai_gib:.0f} GiB terpakai dari {disk_total_gib:.0f} GiB")

    print()
    print(f"=== PERKIRAAN BEBAN {beban['jumlah_node']} NODE SENSOR ===")
    print(f"Jeda pembacaan   : {beban['jeda_detik']} detik")
    print(f"Baris per node   : {beban['baris_per_node']} baris/hari")
    print(f"Total baris/hari : {beban['total_baris']} baris/hari")
    print(f"Volume data/hari : {beban['data_mib']:.2f} MiB/hari")
    print(f"Volume 30 hari   : {beban['data_mib'] * 30:.1f} MiB")

    with open("results/peta.json", "w", encoding="utf-8") as f:
        json.dump({"identitas": ident,
                   "memori_kb": {k: mem[k] for k in
                                 ("MemTotal", "MemAvailable", "MemFree", "Buffers",
                                  "Cached", "SwapTotal")},
                   "cpu_logis": os.cpu_count(),
                   "disk_gib": {"total": round(disk_total_gib, 1),
                                "terpakai": round(disk_pakai_gib, 1)},
                   "beban": beban,
                   "komponen": [{"perangkat": p, "so": s, "jenis": j, "fungsi": f}
                                for p, s, j, f in KOMPONEN]},
                  f, indent=2)
    print("\ntersimpan: results/peta.json")


if __name__ == "__main__":
    main()
```

A.figures.py

```
"""Gambar peta ekosistem SO dan grafik pemakaian memori untuk laporan I-1."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

FONT = "DejaVu Sans"
OUT = "results"


def kotak(ax, x, y, w, h, judul, isi, warna, tepi, ukuran=7.4):
    ax.add_patch(FancyBboxPatch((x, y), w, h,
                                boxstyle="round,pad=0.015,rounding_size=0.05",
                                facecolor=warna, edgecolor=tepi, linewidth=1.2))
    ax.text(x + w / 2, y + h - 0.16, judul, ha="center", va="top", fontsize=8.6,
            fontfamily=FONT, fontweight="bold", color=tepi)
    ax.text(x + w / 2, y + h - 0.44, isi, ha="center", va="top", fontsize=ukuran,
            fontfamily=FONT, linespacing=1.55, color="#222222")


def panah(ax, x1, y1, x2, y2, label=None, warna="#444444", putus=False, geser=0.0):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle="-|>",
                                 mutation_scale=12, color=warna, linewidth=1.1,
                                 linestyle="--" if putus else "-"))
    if label:
        ax.text((x1 + x2) / 2, (y1 + y2) / 2 + 0.09 + geser, label, ha="center",
                va="bottom", fontsize=6.8, fontfamily=FONT, color=warna, style="italic")
```

```
def gambar_peta(d):
    fig, ax = plt.subplots(figsize=(9.6, 4.6))
    ax.set_xlim(0, 9.6)
    ax.set_ylim(0, 4.6)
    ax.axis("off")

    biru, tepi_biru = "#E8EEF7", "#3B5B8C"
    oranye, tepi_oranye = "#FDF2E0", "#B7791F"
    hop = d["identitas"]
    mem = d["memori_kb"]

    ax.add_patch(FancyBboxPatch((0.12, 0.95), 5.9, 3.4,
                                boxstyle="round,pad=0.02,rounding_size=0.06",
                                facecolor="#F9FBFF", edgecolor="#9BB3D4",
                                linewidth=1.0, linestyle="--"))
    ax.text(0.28, 4.16, "LAPIS LAHAN DAN TEPI (EDGE)", fontsize=7.8,
            fontfamily=FONT, color="#5A7599", fontweight="bold")

    ax.add_patch(FancyBboxPatch((6.25, 0.95), 3.2, 3.4,
                                boxstyle="round,pad=0.02,rounding_size=0.06",
                                facecolor="#FFFCF7", edgecolor="#D9B48F",
                                linewidth=1.0, linestyle="--"))
    ax.text(6.4, 4.16, "LAPIS CLOUD DAN MOBILE", fontsize=7.8,
            fontfamily=FONT, color="#A6753F", fontweight="bold")

    kotak(ax, 0.28, 1.15, 1.7, 2.75, "1. Mikrokontroler",
          "ESP32 · 12 node\n\nFreeRTOS\nEmbedded / RTOS\nRAM ~520 KB\nTanpa MMU\n\n"
          "Baca sensor 5 detik\nKendali pompa",
          biru, tepi_biru)

    kotak(ax, 2.42, 1.15, 1.9, 2.75, "2. Gateway kebun",
          "Raspberry Pi OS Lite\nServer (headless)\n\n"
          "Teruskan data sensor\nBuffer saat offline\n\n"
          f"Acuan ukur:\n{hop['hostname'][:19]}\n{hop['distro']}",
          biru, tepi_biru)

    kotak(ax, 4.72, 1.15, 1.15, 2.75, "3. Operator",
          "Komputer\noperator\n\nDesktop\n\n"
          f"{os.cpu_count()} CPU\n{mem['MemTotal'] // 1024 // 1024} GiB RAM\n\n"
          "Alat ukur dan\ndasbor",
          biru, tepi_biru)

    kotak(ax, 6.45, 2.7, 2.8, 1.2, "4. Server cloud",
          "Ubuntu Server LTS · Server\nAgregasi multi-gateway,\nbasis data, TLS",
          oranye, tepi_oranye, 7.2)

    kotak(ax, 6.45, 1.15, 2.8, 1.2, "5. Ponsel pemantau",
          "Android · Mobile\nSandbox UID, notifikasi,\nkendali pompa",
          oranye, tepi_oranye, 7.2)

    # aliran data (penuh) dan perintah (putus-putus)
    panah(ax, 1.98, 2.75, 2.42, 2.75, warna="#2F6B3C")
    panah(ax, 2.42, 2.45, 1.98, 2.45, warna="#A03B2C", putus=True)
    ax.text(2.2, 2.86, "data", ha="center", fontsize=6.4, fontfamily=FONT, color="#2F6B3C")
    ax.text(2.2, 2.22, "perintah", ha="center", fontsize=6.4, fontfamily=FONT,
            color="#A03B2C")

    panah(ax, 4.32, 2.75, 4.72, 2.75, warna="#2F6B3C")
    panah(ax, 4.72, 2.45, 4.32, 2.45, warna="#A03B2C", putus=True)

    panah(ax, 5.87, 3.35, 6.45, 3.35, warna="#2F6B3C")
    panah(ax, 6.45, 3.05, 5.87, 3.05, warna="#A03B2C", putus=True)
    ax.text(6.16, 3.46, "HTTPS", ha="center", fontsize=6.4, fontfamily=FONT,
            color="#2F6B3C")

    panah(ax, 8.6, 2.7, 8.6, 2.35, warna="#2F6B3C")
    panah(ax, 8.2, 2.35, 8.2, 2.7, warna="#A03B2C", putus=True)
    ax.text(8.75, 2.43, "REST", fontsize=6.4, fontfamily=FONT, color="#2F6B3C")

    ax.text(0.28, 0.62, "Hijau: data sensor mengalir ke atas (edge → cloud). "
                        "Merah putus-putus: perintah aktuator mengalir balik.",
            fontsize=7.2, fontfamily=FONT, color="#555555", style="italic")
    ax.text(0.28, 0.34, "Kotak 3 adalah mesin tempat angka pada laporan ini diukur; "
                        "kotak 1, 2, 4, dan 5 adalah perannya pada ekosistem.",
            fontsize=7.2, fontfamily=FONT, color="#555555", style="italic")

    ax.set_title("Peta ekosistem sistem operasi pada smart farming",
                 fontsize=11, fontfamily=FONT, fontweight="bold", pad=4)

    fig.tight_layout()
    fig.savefig(f"{OUT}/gambar-peta-ekosistem.png", dpi=220, facecolor="white",
                bbox_inches="tight")
    plt.close(fig)
```

```
def gambar_memori(d):
    mem = d["memori_kb"]
    total = mem["MemTotal"] / 1024
    tersedia = mem["MemAvailable"] / 1024
    terpakai = total - tersedia
    swap = mem["SwapTotal"] / 1024

    fig, ax = plt.subplots(figsize=(6.6, 2.1))
    ax.barh([0], [terpakai], color="#A03B2C", edgecolor="white", height=0.5,
            label=f"Terpakai {terpakai:.0f} MiB")
    ax.barh([0], [tersedia], left=[terpakai], color="#3B5B8C", edgecolor="white",
            height=0.5, label=f"Tersedia {tersedia:.0f} MiB")
    ax.set_yticks([])
    ax.set_xlim(0, total * 1.02)
    ax.set_xlabel("Memori (MiB)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_visible(False)
    ax.legend(frameon=False, fontsize=7.5, ncol=2, loc="lower center",
              bbox_to_anchor=(0.5, 1.05))
    ax.set_title(f"Pemakaian RAM {d['identitas']['hostname']} "
                 f"(MemTotal {total:.0f} MiB, swap {swap:.0f} MiB)",
                 fontsize=9, fontfamily=FONT, fontweight="bold", pad=30)
    fig.tight_layout()
    fig.savefig(f"{OUT}/gambar-memori.png", dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/peta.json", encoding="utf-8") as f:
        d = json.load(f)
    gambar_peta(d)
    gambar_memori(d)
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```

A.ambil-bukti.sh

```
#!/usr/bin/env bash
# Membuat bukti terminal asli untuk laporan I-1.
# Setiap langkah: ketik perintah ke shell interaktif, tunggu, lalu tangkap
# jendela terminalnya. Prompt, gema perintah, dan keluaran berasal dari bash
# dan dari mesin ini, bukan gambar buatan.
set -u

DIR=${1:?butuh folder kerja}
OUT=${2:?butuh folder keluaran}
FIFO=/tmp/shotest/shot.fifo
mkdir -p "$OUT"
rm -f "$FIFO"; mkfifo "$FIFO"

cd "$DIR" || exit 1

io.elementary.terminal -n -w "$DIR" -x "bash --noprofile --rcfile $HOME/.bashrc -i < $FIFO" \
  > /dev/null 2>&1 &
TERM_PID=$!
sleep 4

exec 3>"$FIFO"

tangkap() {
  local nama=$1
  sleep 0.7
  gdbus call --session --dest org.gnome.Shell.Screenshot \
    --object-path /org/gnome/Shell/Screenshot \
    --method org.gnome.Shell.Screenshot.ScreenshotWindow \
    true false false "$OUT/$nama.png" > /dev/null 2>&1
  echo "  -> $nama.png"
}

ketik() {
  printf '%s\n' "$1" >&3
}

# Bersihkan layar supaya baris peluncuran terminal tidak ikut tertangkap.
ketik 'clear'
sleep 1

# 1. prompt dan folder kerja
```

```
tangkap "ss-1-prompt"

# 2. identitas sistem
ketik 'hostname'
tangkap "ss-2-hostname"

ketik 'uname -srm'
tangkap "ss-3-kernel"

ketik 'grep -E "PRETTY_NAME|VERSION_ID" /etc/os-release'
tangkap "ss-4-os-release"

# 3. perangkat keras
ketik 'lscpu | grep -E "^Architecture|^CPU\(s\)|Model name|Thread|Core"'
tangkap "ss-5-cpu"

ketik 'df -hT /'
tangkap "ss-6-disk"

# 4. memori
ketik 'grep -E "^MemTotal|^MemAvailable|^MemFree|^SwapTotal" /proc/meminfo'
tangkap "ss-7-meminfo"

# 5. layanan dan proses
ketik 'ps -p 1 -o pid=,comm='
tangkap "ss-8-init"

ketik 'systemctl list-units --type=service --state=running --no-pager --no-legend | wc -l'
tangkap "ss-9-jumlah-unit"

ketik 'ps -e --no-headers | wc -l'
tangkap "ss-10-jumlah-proses"

# 6. program pemetaan
ketik 'python3 peta_so_kebun.py'
sleep 3
tangkap "ss-11-program"

exec 3>&-
sleep 1
kill "$TERM_PID" 2>/dev/null
rm -f "$FIFO"
echo "selesai: $(ls -1 "$OUT"/*.png | wc -l) tangkapan layar"
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Saya tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun skrip pengukuran dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka diambil dari keluaran perintah pada tangkapan layar; skrip dijalankan ulang dengan ambil-bukti.sh dan hasilnya dicocokkan dengan Tabel 5.2

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.

☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.

☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☒ Setiap tangkapan layar menampilkan prompt dan memiliki nomor serta keterangan.

☐ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.

☒ Pernyataan penggunaan AI generatif sudah diisi.

☒ Berkas disimpan sebagai PDF dengan nama C_I-1_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Ketepatan konsep	40%	Ya — empat kategori dipakai konsisten di Tabel 5.1 dan Bagian 2
Kedalaman analisis	30%	Ya — tiga poin tugas dijawab dengan kaitan ke MMU dan jumlah pengguna
Bukti praktik/perhitungan	20%	Ya — 11 tangkapan layar asli dan tabel hasil ukur dengan kolom sumber
Orisinalitas dan kerapian	10%	Ya — diagram dibuat dari data ukur, daftar pustaka memakai gaya APA